

# VCF Core: A Semantic Foundation for RDF Representations of VCF

Elias Crum<sup>1,2,\*</sup>, Bart Buelens<sup>2</sup>, Gökhan Ertaylan<sup>2</sup> and Ruben Taelman<sup>1</sup>

<sup>1</sup>IDLab, Department of Electronics and Information Systems, Ghent University – imec, Belgium

<sup>2</sup>Flemish institute for Technological Research (VITO) Mol, Belgium

## Abstract

Medical applications of genomic data depend on connecting variant observations to clinical information and biomedical knowledge. Linked Data offers a framework for this integration, but the Variant Call Format (VCF) does not natively provide an RDF representation of the observations and their interpretation context. We present VCF Core Vocabulary 2.1.2, a shared semantic target for representing VCF files, header declarations, records, annotations and sample data. The vocabulary integrates existing provenance and genomic models while retaining VCF-specific field definitions, ordering and missingness. Expanded sample calls expose individual values, whereas condensed, sample-ordered vectors provide an alternative for cohort data. Version-specific registries and validation profiles address VCF 4.1–4.5, including 122 reserved INFO and FORMAT declarations for VCF 4.5. A curated assessment records representation evidence for all 104 inventoried logical-model constructs and enforcement evidence for 87, and a set of 123 curated alignments documents connections to existing variation models and reference ontologies. We evaluate the vocabulary with executed checks rather than term counts. A specification-derived assessment of 94 requirements over VCF 4.1–4.5 supplies queries and expected answers authored in advance; replaying the 39 requirements that a ten-fixture subset exercises over graphs from a second, separately implemented producer gives agreement on 536 of 572 checks, 508 of them passing for both, with no divergence attributable to a representation gap. A semantic round-trip recovers the fixed columns and genotypes of all 35 records, in the expanded profile, from structured properties alone, and one integration example answers an allele-dependent question under VCF 4.5 local alleles with checked answers and recovered provenance. The replay also found a silent correctness defect in local-allele handling that per-producer testing had not, which we report as evidence for keeping representational mechanism, implementation and correctness apart. We then discuss how this foundation could support reusable conversion engines, traceable links to biomedical resources, and SPARQL queries combining genomic observations with medically relevant information.

## Keywords

Variant Call Format, RDF, Genomic data, Vocabulary, Data integration, SHACL

## 1. Introduction

Healthcare research and personalized medicine depend on connecting clinical observations, molecular data, and biomedical knowledge. These resources are distributed across institutions and databases, where differences in data structures, identifiers and terminology complicate their combined use [1, 2]. Semantic Web approaches provide a basis for addressing this heterogeneity through shared identifiers, explicit relationships and reusable vocabularies [3, 4]. Leveraging these strategies has enabled harmonizing clinical data across institutions while integrating knowledge about genes, diseases, and phenotypes to support biomedical investigations [1, 2]. Meaningful integration of these distributed resources nevertheless depends on retaining the context in which source observations were recorded, not only on adopting common identifiers or terminology [1]. For genomic data, this motivates a reusable semantic representation that preserves source observations and their interpretation context while enabling connections to clinical information and biomedical knowledge.

---

SWAT4HCLS 2027, February 1–4, 2027, Basel, Switzerland

\*Corresponding author.

✉ elias.crum@ugent.be (E. Crum); bart.buelens@vito.be (B. Buelens); gokhan.ertaylan@vito.be (G. Ertaylan); ruben.taelman@ugent.be (R. Taelman)

🆔 0009-0005-3991-754X (E. Crum); 0000-0001-7734-3747 (B. Buelens); 0000-0001-5602-6435 (G. Ertaylan); 0000-0001-5118-256X (R. Taelman)



© 2027 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

The Variant Call Format (VCF) is widely used to exchange genomic variant data [5]. A VCF file can be understood as a structured table in which each data row describes a genomic location and the sequence observed there relative to a reference genome. For each row, a record contains the reference allele, one or more alternative alleles, and optional annotations describing properties such as quality, filtering, or predicted biological effects. When individual samples are included, additional columns record sample-level observations such as the genotype or sequencing depth. A header at the beginning of the file defines many of these fields and specifies how their values should be interpreted [6].

These relationships are important because VCF often encodes meaning indirectly through the structure of the file. For example, the reference allele is assigned index 0 and alternative alleles are numbered in the order in which they appear. A genotype written as 0/2 therefore does not itself identify two DNA sequences: it indicates that the sample carries the reference allele and the second listed alternative allele. Similarly, the meaning and expected number of values for an annotation may depend on a definition given in the file header. This simple example illustrates a broader characteristic of VCF: the format achieves compactness by relying on relationships that are understood within the file rather than stated explicitly with each value. More complex cases, discussed later, depend on several such relationships at once. When VCF observations are integrated with information outside the file, these implicit relationships must remain available if the observations are to be integrated meaningfully with other genomic, clinical, and biomedical data. The challenge is therefore not that VCF lacks contextual information, but that much of this context is encoded through the internal structure of the file rather than exposed as explicit, independently addressable relationships.

A semantic representation offers a way to make these dependencies explicit while preserving their connection to the original observations. Doing so supports semantic interoperability and reuse, central concerns of the Findable, Accessible, Interoperable and Reusable (FAIR) principles [7]. The wider FAIRness of a dataset also depends on its identifiers, metadata, access conditions and provenance, rather than on the choice of file format alone [8]. The Resource Description Framework (RDF) provides a graph-based model of resources and their relationships, while Linked Data principles support the use of shared identifiers and links across datasets [9, 10]. Together, these provide a basis for exposing VCF observations and their interpretation context alongside connections to external biomedical resources, which is necessary for any downstream clinical or research applications [11].

Existing VCF-to-RDF approaches provide important foundations for this work. The Genomic Feature and Variation Ontology (GFVO) represents VCF concepts within a broader model of genomic features and variation, while VCF2RDF presents an isomorphic semantic mapping of the format [12, 13]. HERO-Genomics places VCF files and records within a broader integration model, while the Genome Variation Ontology (GVO) describes variation, including complex structural variants [14, 15]. The GA4GH Variation Representation Specification (VRS) addresses computable variation descriptions and federated identification rather than all source-file details [16]. These complementary purposes motivate a distinction between representing an observation as recorded and linking it to a biological interpretation. Our aim is therefore not simply to establish that VCF can be represented in RDF, but to provide a version-aware representation that combines broad representation of the source format with explicit connections to existing genomic models. Such a target would allow conversion and querying approaches to develop without repeatedly defining application-specific interpretations of VCF structure, as has been done in the past.

Here, we present the VCF Core Vocabulary (*vcfcv*) [17]. The contribution combines source-context preservation, expanded and condensed sample representations, documented alignments, and version-specific artifacts for VCF 4.1–4.5. We assess these capabilities through specification-derived queries with expected answers, a replay across separately implemented producers, semantic round-trip checks and an allele-dependent biomedical integration example with recovered provenance. These assessments distinguish the existence of a representational mechanism from its implementation and the correctness of the resulting data. We then discuss how this foundation could support reusable conversion engines and integrated, medically relevant genomic querying. Throughout, *vcfc:* denotes <https://w3id.org/vcf-core/vocab#>.

## 2. Methods

### 2.1. Assessment objectives, evaluated artifacts, and essential model

The assessment undertaken in this study addressed three questions. **RQ1:** Does VCF Core preserve the intended meaning of the VCF constructs it claims to support? **RQ2:** Do the version-specific artifacts and alternative sample profiles behave as documented? **RQ3:** Does the resulting representation support a reproducible integration task while retaining source context? **RQ4:** How does VCF Core relate to existing VCF semantic models?

We address RQ1 and RQ2 by testing the vocabulary against requirements drawn from the VCF specifications followed by checking whether the same expected results are obtained from converter-generated RDF. A separate biomedically-motivated integration case study addresses RQ3. To address RQ4, we conduct a comparative representation assessment of existing semantic models that model VCF contents and identify how they relate to the terms provided by VCF Core.

**Evaluated artifacts.** The assessment used VCF Core 2.1.2 together with its accompanying vocabulary files, version-specific definitions, validation rules, mappings and test data [17]. The first set of checks was run on RDF generated directly from these test materials. We then repeated the relevant checks using RDF produced by VCF-RDFizer [18]. For the results reported here, we used VCF-RDFizer 3.0.3. The producers share an author but not implementation code; this is an implementation cross-check, not independent third-party reproduction.

**The evaluated converter release.** Every figure reported here comes from VCF-RDFizer **3.0.3** (be658a2), run from the published image pinned by digest sha256:31f1361b. It is a release rather than a working commit, and the digest rather than the tag is what fixes it, so a moved tag cannot change what was run. Requirements that diverge between the two producers are listed individually with their per-check outcomes in the run's result files, so no total has to be inferred from another.

**Reproducing any of this.** One command regenerates every experimental figure in this paper:

```
sh reproduce/run.sh
```

It fetches the vocabulary at v2.1.2 and both converter releases at their tags, pulls the two container images by digest, converts the ten fixtures in both sample profiles with each release, runs every analysis, and writes one self-contained run directory. That directory holds the machine description and toolchain versions (`env.json`), every command executed with its exit code and duration (`steps.jsonl`), the per-step output (`logs/`), the converted graphs each result was computed from (`graphs/`), the result files (`results/`) and a summary. The run reported here took 616 seconds over 15 steps with no failures.

Provenance that a reader would otherwise have to reconstruct is recorded by the run itself: the pinned specification revisions with their SHA-256 digests, the fixture set, the SPARQL engine that executes every coverage, round-trip and integration query (rdflib 7.6.0), the SHACL validator (pySHACL) with the ontology graph loaded and RDFS inference enabled, and the container image digests. The SHACL configuration is not incidental — without the ontology graph and RDFS inference, `sh:class` constraints cannot see the vocabulary's subclass axioms and conforming data reports violations, so the validator is run the way the vocabulary's own test suite runs it. Earlier, incremental runs are archived separately and are cited by nothing here.

**Representation needed for the assessment.** VCF Core retains a file, its header declarations, records and sample observations as distinct resources (Figure 1). A `VariantCall` groups the observations associated with a record rather than identifying an additional source-file line. File-scoped identifiers and provenance distinguish observations even when they concern the same alteration. A `VCFSample` identifies a source column, not automatically a patient or specimen. Direct reuse of PROV and DCAT

supports source descriptions, while SO and FALDO provide connections to alteration and location models [19, 20, 21, 17]. These connections do not perform variant normalization or establish instance identity across files.

Field definitions are separate from their occurrence in a file header, and values can link to the declaration needed to interpret them. Lexical values, typed accessors and explicit missingness remain distinguishable; ordered allele resources retain the relationship between allele indices and sequence alternatives. These mechanisms are the objects of assessment, rather than evidence of correct conversion by themselves.

The expanded profile exposes individual sample calls and FORMAT values through graph patterns. The condensed profile stores sample-ordered vectors with an explicit definition, encoding and payload; the `VCFTxtVector` encoding retains tab-separated values in sample order. Both retain file, header and record context, but accessing an individual condensed cell requires decoding. VCF 4.1–4.5 share the model while selecting version-specific registries and validation overlays from the file’s declared version [22, 6, 17]. This separates representation choices from the version-dependent rules tested below.

--

### 3. VCF as a semantic representation problem

#### 3.1. What a VCF file describes

A VCF file contains meta-information lines, a column-header line, and data records. Meta-information lines beginning with `##` identify the format version and can describe the reference sequence, genomic sequence segments called contigs, and the fields used in the records. The column header begins with `#CHROM`. Each subsequent record describes a genomic position, the reference sequence at that position and one or more alternatives, together with observations and annotations [6]. See [5] for a detailed description of the format, and see [23] for a discussion of the FAIR implications of VCF formatting. Table 1 summarizes the main components.

**Table 1**

VCF components and the interpretation context that an RDF representation needs to retain. INFO describes a record; FORMAT describes the contents of sample columns.

| Component           | Role in the source file                                                                                                              |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Header declarations | Define field identifiers, types, cardinalities and descriptions, together with reference and processing context.                     |
| CHROM, POS, ID      | Identify the reference contig, position and optional record identifiers. Position alone is not a globally unique variant identifier. |
| REF, ALT            | Give the reference allele and ordered alternative alleles. An allele is a sequence alternative at the represented locus.             |
| QUAL, FILTER, INFO  | Describe call quality, filter status and record-level annotations, including producer-defined fields.                                |
| FORMAT and samples  | Declare an ordered set of keys and give corresponding values for each sample, such as genotype (GT) and read depth (DP).             |

#### 3.2. Why version and source context matter

VCF format has been evolving since its first public release in 2011. At the time of writing, the specifications are maintained by the Global Alliance for Genomics and Health (GA4GH) [24] with versions 4.1–4.5 hosted on a GA4GH-maintained github repository [25]. The most current specification is VCF v4.5 and was released February 25, 2026 [6]. VCF 4.1–4.5 share the basic structures described above but differ in supported declarations and constraints [22, 6]. For example, later versions introduce new ways to describe variants and their associated metadata, while also refining how existing fields should be interpreted. To further complicate format versioning, the observations recorded in a VCF must remain

distinguishable from the biological entities they describe. Two files may report the same alteration with different annotations or quality estimates, just as a sample column may refer to a patient specimen, a pooled sample, or a synthetic example. A common representation should therefore preserve both version-specific semantics and the provenance of each observation, without automatically equating VCF records with normalized variants or sample columns with patients.

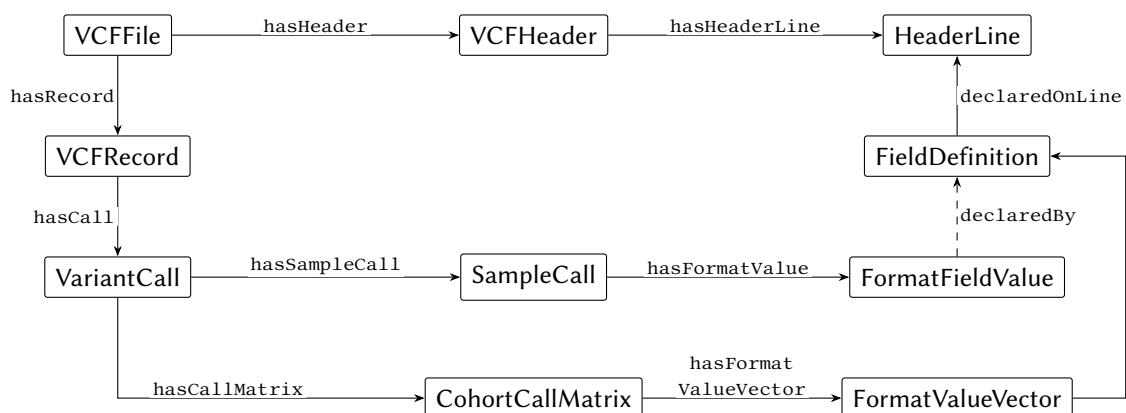
## 4. The VCF Core vocabulary

### 4.1. Retaining observations and their provenance

VCF Core separates a `VCFFile`, its `VCFHeader`, individual `VCFRecord` resources and a `VariantCall` associated with each record. The call groups quality, filter, annotation and genotype information; it is a modeling abstraction rather than an additional line in the source file. Figure 1 shows this shared structure and the two sample representations.

The file specializes `dcap:Distribution` and `prov:Entity`, while records and calls provide further attachment points for provenance [19, 17]. File-based IRI templates preserve the relationship between source, record and field. Because a VCF identifier can be missing or non-unique, a producer must use a record identifier that distinguishes rows within the source. This approach allows observations from different files to remain separate even after they have been linked to a common biological entity.

The property `asSequenceAlteration` connects a record or call to a separately described Sequence Ontology (SO) sequence alteration [20]. FALDO supplies location, reference and position descriptions, including uncertain positions where appropriate [21]. Thus, VCF Core does not introduce a competing general-purpose location model. Establishing cross-file links still requires compatible reference sequences and correct allele interpretation; the vocabulary provides the connection points rather than a normalization algorithm.



**Figure 1:** Selected VCF Core relationships; prefixes are omitted. Both sample profiles retain the file, record and header context. An expanded value can link to its definition through `declaredBy`; the equivalent relation is required for a condensed vector. Sample identity and ordering are described separately through `VCFSample` and `SampleSet`.

### 4.2. Making field interpretation explicit

Header declarations are represented because they determine how values should be read: VCF Core gives each meta-information line type its own class and exposes a field definition’s identifier, Number, Type, Description and optional source and version as data, with generic `HeaderAttribute` resources retaining producer-defined additions [17].

Two modelling decisions in this area carry most of the weight. First, a standardized definition and its occurrence in a file are kept apart — the reserved `FORMAT/DP` definition describes a key the specification supplies, while a header line records what one file declares — so `FieldDefinition` and

HeaderLine are distinct resources joined by declaredOnLine, and values reach their declaration through declaredBy. Without that separation a registry entry would be forged into every file that never declared it. Second, lexical and parsed forms are both retained: fieldValue keeps the source token or list while optional integer, decimal and boolean properties serve scalar queries, so a missing depth is never coerced to zero and a non-finite float is never forced into xsd:decimal. Standalone missing values use vcfc:Null, and ordered allele resources index REF at zero and ALT upwards, which is what makes a genotype integer and an allele-dependent annotation resolvable. Raw source text may be retained alongside, but a preserved string is never counted as structured coverage.

### 4.3. Choosing how sample data are represented

The expanded profile connects each call to per-sample SampleCall resources and individual FormatFieldValue resources. This makes values directly accessible through graph patterns and allows an application to annotate a particular sample observation. A VCFSample identifies the source column; linking it to a biological specimen or patient is a separate integration task. Expanded representation is applicable to both single- and multi-sample files when direct access justifies the additional graph structure.

Condensed representation associates each call with a cohort matrix. The CohortCallMatrix groups one FormatValueVector for each represented FORMAT key. A shared SampleSet identifies samples by name and one-based index. Each vector specifies its definition, encoding and payload. The initial VCFTextVector encoding stores tab-separated sample values in index order, including missing values. The synthetic example therefore produces GT values 0/1, 0/0, ./ and DP values 42, 18, . in corresponding positions.

This is a change in graph granularity, not a replacement of sample observations by cohort summaries. Individual condensed cells require decoding before per-sample comparisons; they are not directly bound by ordinary basic graph patterns. For  $R$  records,  $S$  samples and  $F$  FORMAT keys, expanded materialization creates  $RS$  sample calls and  $RSF$  value resources. Condensed materialization creates  $S$  sample resources, one sample set,  $R$  matrices and  $RF$  vectors. These counts exclude shared metadata and do not establish savings in bytes, memory or query time. The underlying payload still contains  $RSF$  values.

Listing 1 illustrates direct expanded access. On the supplied synthetic graph it returns SAMPLE1 with depth 42; decoding the condensed graph gives the same selection and recovers the same six FORMAT cells. The threshold is illustrative, not a clinical quality standard. This bounded example establishes neither general conversion fidelity nor clinical validity.

Listing 1: An expanded-profile query over the supplied synthetic example.

```
PREFIX vcfc: <https://w3id.org/vcf-core/vocab#>
SELECT ?sampleName ?depth WHERE {
  ?file vcfc:representationProfile vcfc:ExpandedRepresentation ;
        vcfc:hasRecord/vcfc:hasCall/vcfc:hasSampleCall ?call .
  ?call vcfc:forSample/vcfc:sampleName ?sampleName ;
        vcfc:hasFormatValue ?value .
  ?value vcfc:declaredBy/vcfc:fieldId "DP" ;
        vcfc:fieldValueInteger ?depth .
  FILTER (?depth >= 20)
}
```

## 5. Building on existing semantic models

### 5.1. Complementary contributions

The usefulness of VCF Core depends on its relationship to the models already used for genomic data. GFVO was developed from several genomic formats, explicitly including VCF 4.1 and 4.2 [12]. VCF2RDF demonstrated source-preserving semantic conversion [13]. HERO-Genomics includes VCF files and records within a broader integration model, while the Genome Variation Ontology (GVO) addresses detailed variation descriptions, including complex structural variants [14, 15]. The GA4GH Variation Representation Specification (VRS) provides computable variation descriptions and federated identification, rather than a representation of every source-file detail [16]. These are complementary purposes, not successive attempts at the same task.

To place VCF Core among them without asserting superiority, we inspected the published artifact of each model and scored nine retrieval criteria drawn from the interpretation context a VCF file carries about itself (Table 2). Each criterion is a question about what can be retrieved, not a judgement of quality, and each verdict names the term or absence it rests on. GA4GH VRS is deliberately not scored: it describes computable variation identity rather than a source file, so scoring it against file-format criteria would manufacture a contrast its authors never sought.

**Table 2**

What each inspected artifact lets a consumer retrieve. **E** explicitly represented; **C** representable with additional conventions; **S** outside the model’s stated scope, which is not a defect. Scored against GFVO (BioInterchange master), HERO-Genomics 1.0, GVO (versionInfo 2021-11-18) and the VCF2RDF term set (v4\_2), GFVO and HERO-Genomics retrieved 2026-09-12, GVO and the VCF2RDF term set 2026-09-13.

| Retrieval criterion                              | VCF Core | GFVO | HERO | GVO | VCF2RDF |
|--------------------------------------------------|----------|------|------|-----|---------|
| A record traces to its source file               | E        | E    | E    | S   | C       |
| A declaration exposes ID, Number, Type           | E        | S    | S    | S   | E       |
| The file’s VCF version is readable               | E        | S    | S    | S   | C       |
| ALT allele <i>n</i> is addressable by index      | E        | C    | C    | S   | C       |
| Ploidy, calls and phasing are separable          | E        | C    | C    | S   | C       |
| A (sample, field) value without splitting a cell | E        | C    | C    | S   | C       |
| A Number=A/R/G item binds to its allele          | E        | C    | C    | S   | C       |
| An explicit . differs from “not stated”          | E        | S    | C    | C   | C       |
| The sample encoding is readable from the graph   | E        | S    | S    | S   | S       |

Two contrasts explain the pattern. HERO-Genomics and GVO both retain the INFO column as a single literal — our alignment set records `hero:vcfInfo` and `gvo:info` as exact matches to `vcfc:infoRaw` — so every INFO-derived question requires re-parsing that string and supplying the cardinality rule from outside the graph. GFVO makes the opposite choice, modelling particular field *meanings* as first-class concepts (`AlleleCount`, `Coverage`, `MappingQuality`), which answers those fields well and leaves the declaration mechanism unaddressed. VCF2RDF is the instructive case: its published term set does define `Number`, `Type` and `Description` as dereferenceable terms, so the cardinality rule is named rather than left outside the vocabulary — but with no `allele-index` term a consumer must still split the list and count ALT alleles by re-reading a literal. Having the declaration is necessary and, alone, insufficient; binding a value to an allele is the part that must be represented rather than inferred.

These verdicts are bounded in three ways. The criteria were chosen for the interpretation problem VCF Core addresses, so this is not a neutral ranking — a matrix chosen by GVO’s authors would look different and would be equally legitimate. GVO’s column is S in every row but missingness, because the retrieved ontology declares 48 classes, 12 datatype properties and *no* object properties: a model of variation types, doing something other than file modelling. Missingness scores C there only because a literal-valued property can carry a dot, while distinguishing that from an absent triple remains a

convention the ontology does not supply. The VCF2RDF column rests on a published term set carrying specification glosses rather than RDF axioms, and on no converter output, so what its graphs contain is still taken from its paper. Finally, “requires additional conventions” states what an inspected artifact establishes, never that a model’s community could not represent something.

## 5.2. Reuse and curated alignment

Direct reuse and cross-model alignment are kept distinct. PROV, DCAT, SO and FALDO terms contribute to the core representation through axioms or instance descriptions. The accompanying alignment set documents correspondences to models whose assumptions do not always coincide with a source-oriented VCF representation. Table 3 summarizes the 123 mappings from 65 VCF Core terms to 103 external terms [17].

**Table 3**

Curated alignment targets. Counts describe the mapping set, not comparative specification coverage or demonstrated instance-level interoperability.

| Target                 | Mappings | Connection to VCF Core                                            |
|------------------------|----------|-------------------------------------------------------------------|
| GVO                    | 39       | Variation and VCF-related properties.                             |
| HERO-Genomics          | 17       | File, record and observation descriptions.                        |
| GA4GH VRS 2.0          | 15       | Computable descriptions of variation.                             |
| GFVO                   | 14       | Genomic features and field-related concepts.                      |
| Med2RDF                | 3        | VCF-oriented allele properties.                                   |
| SO, ChEBI, GENO, FALDO | 35       | Alteration types, modifications, genotype concepts and locations. |

Mappings are curated in the Simple Standard for Sharing Ontological Mappings (SSSOM), which records the mapping predicate, justification and associated metadata [26]. A generated Turtle bridge provides the RDF form. Of the 123 mappings, 48 are also asserted inline in ontology modules; the remaining 75 are available through the separately loaded bridge. Keeping this distinction visible lets consumers identify which mapping assertions accompany the core and which require an additional module.

The set contains 57 `skos:exactMatch`, 40 `skos:closeMatch`, 16 `skos:relatedMatch` and 10 `skos:narrowMatch` assertions. These do not establish OWL class or property equivalence, nor do they generate the instance links needed for a biomedical query. They nevertheless have defined semantics: `skos:exactMatch` is transitive, whereas `skos:closeMatch` is not [27]. Consequently, mixed mapping paths cannot be treated as unrestricted rules for predicate substitution, and mappings between different VRS releases require explicit review.

The repository additionally preserves 116 SSSOM mappings from work in the 2025 SWAT4HCLS BioHackathon separately from VCF Core’s own assertions [28]. Of 47 terms previously recorded as lacking a VRS counterpart, eight have a source-oriented VCF Core counterpart, including file, quality, and as-written allele concepts. This illustrates the contribution of retaining source semantics alongside biological models; it does not mean that the 39 remaining annotation and interpretation concepts should all become VCF Core terms.

## 6. Specification coverage and implementation evidence

### 6.1. One representation with version-specific rules

VCF Core uses one shared model for common structures and version-specific registries and validation overlays for VCF 4.1–4.5 [17]. The registries contain 40, 40, 67, 78 and 122 explicit reserved declarations, respectively. For VCF 4.5 these comprise 51 INFO and 71 FORMAT definitions, including three entries for parameterized base-modification key families. Prose-only fields in older versions are not assigned Number or Type constraints absent from their specifications.

The overlays condition rules on the owning file’s declared version. Thus, the CIPOS example in Section 3 can be assessed under the appropriate historical or current rule, while the LA, LR, LG and M cardinalities are restricted to VCF 4.5. Shared validation is divided into portable SHACL structure checks, SPARQL-based cross-resource checks and raw/parsed-value consistency checks [29, 17]. This organization permits different validator capabilities without implying that each layer independently validates the entire format.

## 6.2. What the evidence establishes, and at what strength

Coverage claims are easy to overstate because three different things are routinely reported as one number. We keep them apart. *A mechanism exists* when a term, rule or documented pattern is available; this establishes nothing about correct instantiation. *An example instantiates it* when a fixture demonstrates the mechanism in use; this establishes nothing about general correctness. *Behaviour is independently checked* when an executed query agrees with an expected answer established separately from the code that produced the graph. Only the third supports a correctness claim, and only within what was tested.

Two assessments and two cross-checks address three questions: whether VCF Core preserves the intended meaning of the constructs it claims to support (RQ1); whether the version-specific artifacts and the two sample profiles behave as documented (RQ2); and whether the result supports a reproducible integration task that retains source context (RQ3).

**Mechanism.** The curated VCF 4.5 inventory enumerates 104 logical constructs across file and header organization, declarations, types and cardinalities, lexical rules, fixed fields and alleles, genotypes, structural variation, repeats and gVCF. All 104 have preserved, structured representations and 87 additionally have an enforcement rule. Separately, all 333 reserved Number/Type rows extracted from the specification texts match the versioned registries [17]. Those 333 rows — 31, 31, 69, 79 and 123 per version — are the keys each source declares explicitly in ##INFO=##FORMAT= blocks or reserved-key tables, and are a different population from the registry entries counted above. VCF 4.1 and 4.2 each define a further 29 reserved keys in prose bullets that give no Number and, for INFO, no Type; those rows are inventoried but cannot be compared, so this agreement figure is weighted towards VCF 4.3–4.5. The inventory was curated by the vocabulary’s developers, so it maps what its authors listed and cannot reveal an omission nobody listed.

**Instantiation and independent checking.** The specification-derived assessment reads information requirements from pinned VCF 4.1–4.5 sources and tests them against fixtures. It comprises 94 requirements over 210 cases; each case pairs a SPARQL query with an expected answer authored from the specification text *before* the query was run, and every passing query is re-run against an emptied graph as a negative probe. Executing every case on both sample profiles yields 840 nominal query runs, but 165 cases score the identical query and expected answer on two axes, so the independent figure is **465 distinct executions**. Coverage is reported per version because requirements apply per version, and an untested requirement scores zero rather than being excluded (Table 4).

A previous version of this assessment reported 491 reviewed requirements of which 133 had complete evidence. That generation has been superseded: requirements are now scored per version and per profile, and no single figure corresponds to the earlier 133.

## 6.3. Whether the claimed coverage survives a second producer

The assessment above builds its witness graphs with the vocabulary repository’s own materializer, whereas the implementation this paper cites is a separate program, and nothing had established that a query and expected answer written against one also holds over the other — the gap between “the vocabulary can express this” and “the software people will use does”. The two programs share an author but no code: one materializes graphs from a Python fixture builder, the other from RML mappings and

**Table 4**

Reported evidence, its denominator and the claim it supports. The denominators count different things and must not be combined into a single percentage.

| Assessment                                   | Result                            | Supported interpretation                                                                   |
|----------------------------------------------|-----------------------------------|--------------------------------------------------------------------------------------------|
| Curated VCF 4.5 inventory                    | 104/104                           | All inventoried logical constructs have preserved, structured representations.             |
| Enforcement within that inventory            | 87/104                            | A stated requirement is checked for these constructs.                                      |
| Reserved Number/Type rows                    | 333/333                           | Extracted declarations match the version-specific artifacts.                               |
| Requirements with a passing test, by version | 32/69, 34/72, 35/74, 38/83, 51/91 | Conservative demonstrated coverage for VCF 4.1–4.5; untested requirements score zero.      |
| Distinct executed checks                     | 465                               | Query runs that are not a repeat of an identical check.                                    |
| Cross-producer replay                        | 508/572                           | Checks on which the vocabulary’s materializer and the cited converter agree and both pass. |
| Semantic round-trip                          | 35/35 records                     | Fixed columns and genotypes recovered from structured properties alone.                    |
| Recorded full validation run                 | 21 fixtures                       | These examples pass the implemented validation layers.                                     |

streaming emitters. Independence here therefore means independent implementations, not independent authorship, and the check is correspondingly weaker than third-party reproduction.

We replayed the existing queries, against the existing reviewed expected answers, over graphs produced by VCF-RDFizer: ten fixtures spanning VCF 4.1–4.5 and the difficult decisions above, each converted in both profiles, giving 572 checks over 143 cases and 39 requirements. The producers agree on 536 checks and both pass on 508; 34 requirements agree on every check and 28 of those pass throughout. Every check they agree on by both failing is a condensed-profile structure query, the documented storage trade-off.

The check earned its keep during development, when an earlier converter bound Number=LA and Number=LR items to allele indices positionally, ignoring the sample’s LAA subset: a depth list of 20, 30, 10 under LAA=2, 4 landed on alleles 0, 1 and 2 instead of 0, 2 and 4 — plausible, well-formed values on the wrong alleles, which no existing test caught because every existing test ran against the other producer. Release 3.0.3 resolves local alleles correctly, and the requirements exercising them pass for both producers here. Five requirements still diverge: a QUAL datatype the vocabulary deliberately leaves open, three structured accessors the converter has not built (one of them the condensed-vector decoding path), and one empty-list encoding convention. None is a case of the vocabulary being unable to express something — across all 572 checks, zero divergences trace to a representation gap.

A complementary check recovers logical content rather than comparing query answers. Across the same ten fixtures we reconstruct CHROM, POS, ID, REF, ALT, QUAL, FILTER and the GT subfield from the RDF alone: REF and ALT from ordered alleleIndex/alleleValue resources, GT from ordered GenotypeAlleleCall resources with their phasing status, so no untouched source string serves as a shortcut. All 35 records are recovered, with three bounds, all by design: INFO and the non-GT FORMAT subfields are out of scope; the check runs on expanded-profile graphs only, since decoding a condensed vector needs the accessor requirement R21 identifies as missing; and in the local-allele fixture, where each site is written once locally and once globally, matching on position confirms one recovery per site rather than both. It is not whole-file reconstruction.

**Where the two sample profiles agree.** Because every case runs in both profiles, the profile claim can be stated precisely rather than asserted. The profiles return the same outcome on 252 of 286 comparable checks, and the 34 disagreements fall on nine requirements, every one reading a FORMAT-scoped value: genotype structure, trailing-field missingness, sample-local allele and base-modification items, and repeat components. Record-level file, header, declaration and allele questions agree on every check in both profiles. The supported claim is therefore that the profiles are equivalent for file, header,

record and record-level allele questions and divergent by design wherever a query walks per-sample resources. Two of the nine are local-allele requirements, so this paper’s headline feature is among those a condensed graph answers less completely.

**Answers to the three questions.** RQ1 is answered for the 53 of 94 requirements with at least one executed case, the great majority passing. RQ2 is answered for 17 of the 36 version-conditioned requirements, with the profiles behaving as documented: equivalent above the sample level, divergent below it. RQ3 is answered for 27 of the 38 requirements on the provenance chain an integration query must traverse, which Section 6.5 then exercises end to end. None of these fractions is a conformance claim; each is the share of requirements whose behaviour has been checked at all.

**Remaining gaps, classified.** Of the requirements exercised, none reveals a representation gap. One correctness defect was found and fixed; three requirements await structured accessors in the converter; one reflects an interpretation the vocabulary deliberately permits both ways; one is an encoding convention. The largest limitation is absence of evidence rather than negative evidence: 41 of the 94 requirements have no test case yet, concentrated in structural-variant fields with version-specific semantics. Allele-dependent values, this paper’s central modelling claim, are tested for 4 of the 11 requirements that bear on them — adequate to demonstrate the mechanism and to catch the defect above, not adequate to claim general correctness. Outside that classification, four validator findings remain from the release’s own conformance checks: custom structured-header IDs can be absent or duplicated without rejection; numeric base-modification keys can be accepted with incorrect Number/Type pairs; a partially missing phase-set-list example is rejected under an unresolved interpretation; and mixed LF/CRLF line endings are rejected under an unconfirmed reading of the specification. Byte-level checking is reported separately, and neither BCF binary layout nor byte-for-byte reconstruction is a target of the logical model.

## 6.4. Implementation and reproducibility

The release provides OWL/Turtle modules, SHACL profiles, examples, versioned registries, SSSOM mappings and documentation [17]. VCF-RDFizer supplies a separate implementation: release 3.0.3 constructs file and record context with RML mappings and produces expanded or condensed sample structures through streaming emitters [18]. The Section 6.2 assessment was produced with VCF Core 2.1.2 and the vocabulary’s own materializer, with no converter involvement; all converter-dependent results (Sections 6.3–6.5) used VCF-RDFizer 3.0.3, the release that carries the corrections the cross-producer check prompted. Both are tagged, so every figure below comes from a released artifact rather than a working branch. Converting the Section 4.3 example emits 242 expanded and 168 condensed triples, drawing on 93 and 80 distinct VCF Core terms respectively. Every one of those terms is declared in VCF Core 2.1.2, and both graphs conform, with no violations, to the shared and the VCF 4.5 SHACL profile when the shapes are loaded with the vocabulary and RDFS inference, as the vocabulary’s own validation does. The depth query in Listing 1 returns its expected single row against the expanded graph. The converter pins no vocabulary version, so this relationship is established by the behaviour recorded here rather than by a declaration, and it is evidence for the terms these graphs exercise, not for all 467 declared terms or for every historical VCF version.

Every figure in this section comes from one execution of a single script, which fetches the vocabulary and both converter releases at pinned revisions, pulls their container images by digest, converts the fixtures, runs each analysis, and records the machine, the toolchain versions, and every command with its exit code and duration alongside the results. One limitation remains: the toolchain inside the image is not independently pinned, so the image digest, not a list of component versions, is what fixes it. The counts above are converter output; the hand-authored illustration file in the release is smaller, because it omits the allele resources and per-sample genotype and allele-call resources that the converter materializes.

The recorded validation run accepts 21 fixtures and includes a negative control, alongside a regression suite of 101 tests. The published workflow recomputes the reported counts and checks evidence references against the maintained artifacts, and each result in Section 6.2 has a script that reproduces it from the pinned inputs. Review of coverage judgements and mappings nonetheless remains a human task. The established result is full representation of the curated VCF 4.5 inventory with version-aware support across VCF 4.1–4.5, demonstrated to be reproducible with an independent converter on the subset tested — not complete conformance.

## 6.5. One integration example, with checked answers

The claims above concern representation and query behaviour within the VCF world. To test whether the retained context survives contact with an external annotation, we ran one small end-to-end example and checked its answers against expectations authored in advance.

The example is bounded to a single difficulty: allele-dependent values under VCF 4.5 local alleles. LAD is declared `Number=LR`, meaning one value per allele of *the sample's own* local allele subset named by LAA, not per entry of the record's ALT column. Answering “how deep is this alteration in this sample?” therefore requires the declaration, the sample's LAA and the allele's ordinal simultaneously. The input is a pinned four-site, single-sample VCF 4.5 fixture, each site written twice, once with local alleles and once globally; the annotations are a local, explicitly synthetic snapshot of five flagged alterations, labelled as such in the file, and local rather than federated so the example returns the same answer on every run. The link is made in the query rather than by pre-minted identifiers: an annotation names an alteration by contig, position, reference and alternate allele, and the query joins on those four literals. Three assumptions are recorded because they are what would break the link — a shared assembly, as-written allele identity with no normalization or VRS matching, and that a VCF sample column is a sample and not a patient.

Asking which flagged alterations have supporting read depth of at least 20 returns two rows, identical to the expected answers: `chr1:1 G>C` at depth 30 and `chr1:2 A>T` at depth 25, each traced to its file, record, sample and the FORMAT declaration that gives the field its cardinality. The three flagged alterations absent from the answer are each absent for a stated reason rather than by oversight.

The example is worth reporting because of what a wrong implementation would do with it. Reading `Number=LR` positionally — taking the *n*th LAD value as belonging to the *n*th ALT allele, rather than to the *n*th member of the sample's own LAA subset — returns *two rows again*, but `chr1:1 G>A` at depth 30 and `chr1:2 A>C` at 25. Those two alterations and those two depths are named in the expected answers as the positional-reading outcome, derived from the fixture text before any query ran. A check comparing result counts cannot tell the two apart. This is the concrete form of the paper's argument: retaining the declaration, the local allele set and the allele ordinal is not bookkeeping, because dropping it yields a plausible, well-formed, wrong answer delivered with complete provenance.

What this example demonstrates is that the representation carries the interpretation context, that a mapping rule connects it to external annotation, and that the answers match those authored beforehand. What it does not demonstrate is equally clear: the input is synthetic and tiny, the annotations are invented and carry no biological or clinical meaning, the link assumes as-written allele identity, and a single fixture establishes correct local-allele handling here rather than in general.

## 7. Discussion

### 7.1. Connecting source observations to biomedical knowledge

VCF Core can make more of the evidence already present in a VCF file available to semantic applications. It retains observations and their interpretation context, while other models describe specimens, patients, diseases, treatments and biological variation. These layers can develop independently when their relationships are explicit.

Two laboratories reporting the same alteration in separate files could, after reference-aware normalization, link to a common variation description while each retains its own quality values, sample identities and provenance — so a disease association attaches to the description once rather than being copied into every source record, and a revised interpretation can be retrieved alongside the original observations. PROV, FALDO and the published alignments are starting points; the normalization and instance mappings themselves remain to be implemented and evaluated.

Sample identity needs the same separation. In a tumour/normal analysis two columns may be different specimens from one patient, and treating column names as patient identifiers destroys that distinction. An integration layer can link each `VCFSample` to its specimen and both specimens to the patient through a biomedical model such as HERO-Genomics [14]; VCF Core preserves the source side without replacing the clinical one.

## 7.2. A shared target for conversion engines

A reusable vocabulary also separates conversion strategy from representation decisions. A clinical site converting one sample for inspection and a research group converting a cohort can choose different sample profiles — expanded values for direct annotation, condensed vectors with selective expansion for scale — while retaining the same file, header and record model rather than growing incompatible vocabularies per workload. A pipeline spanning VCF 4.2 and 4.5 data can likewise select the registry and validation overlay from `fileFormat` instead of silently imposing the newest cardinality on an older declaration.

That suggests an evaluation order for future work: compare recovered source values and query answers first, as Section 6.3 does, and only then measure conversion time, peak memory, serialized size and loading cost. Section 4.3 is explicit that the profiles trade query granularity rather than demonstrated storage savings, and the present results do not establish which implementation is most efficient.

## 7.3. SPARQL questions that combine genomic and clinical evidence

Section 6.5 established the pattern at the smallest scale that requires it: an external annotation joined to sample-level evidence, with the answer depending on interpretation context that the representation retains. The questions below extend that pattern to clinical settings. They are proposed research scenarios and *not* evaluated workflows: unlike the example in Section 6.5, none has been executed, and each requires additional biomedical data, reviewed instance mappings and appropriate access controls. SPARQL provides the query structure; it does not supply missing interpretation or establish that a returned result is clinically actionable.

**Three questions.** A *pharmacogenomic* review asks which patients with an active clopidogrel prescription have an externally established CYP2C19 phenotype relevant to a guideline for their indication, and which source observations support that assessment; the CPIC guideline shows why phenotype and indication both matter [30], and a workflow such as PharmCAT separates variant processing, allele matching and clinical interpretation [31] — steps that must not collapse into reading one genotype token as a metabolizer phenotype. A *rare-disease* investigation asks which candidate alterations lie in genes associated with diseases sharing the patient’s recorded phenotypic features, joining specimen-linked calls to normalized alterations, gene associations and phenotype descriptions such as HPO [32]; its governing constraint is that a row absent from a variant-only VCF is not evidence that a parent is homozygous reference. An *oncology* review asks which alterations detected in a tumour have curated therapeutic evidence for the patient’s cancer type, joining a resource such as CIViC [33] to normalized alterations and retrieving tumour and normal calls through their specimen links; a missing normal call likewise remains missing evidence, not proof of tumour specificity.

Each reaches its sample-level evidence through the same declaration-to-value path that Section 6.5 executes, and each needs clinical assertions from an application schema that is not part of VCF Core.

The vocabulary retains the observations; completeness, segregation and therapeutic interpretation require evidence it does not carry.

These scenarios could be implemented over a materialized graph or, where source interfaces and authorization permit, through federated SPARQL as demonstrated in related work [34]. Availability of RDF and vocabulary alignments alone does not guarantee complete or performant answers. Future experiments should evaluate answer correctness, instance-link accuracy, provenance recovery and source availability before making claims about medical utility or query speed. The opportunity is to investigate these questions against a common representation instead of repeatedly reconstructing VCF semantics within each application.

## 8. Conclusion

VCF Core provides a shared semantic foundation for representing VCF observations together with the context needed to interpret them. It combines reuse of existing models, explicit source structure, alternative sample representations and version-specific artifacts for VCF 4.1–4.5. All 104 constructs in the curated VCF 4.5 inventory are represented, and the claims that matter have been executed rather than asserted: the specification-derived checks agree across a second producer on 536 of 572 replayed checks with no divergence traceable to the vocabulary, all 35 records round-trip through structured properties, and one integration example returns answers authored in advance together with their source provenance.

The remaining work is specific rather than general. Evidence is absent for 41 of the 94 requirements, concentrated in structural-variant fields, and thinner than we would like for allele-dependent values in particular; three structured accessors are not yet implemented in the cited converter; one QUAL datatype question is an interpretation the vocabulary deliberately leaves open; and the per-sample condensed profile cannot yet answer the hardest missingness question. None of these is a representation gap, and none is hypothetical — each is a named requirement with a recorded result.

What this foundation makes practical is the step from representing genomic files to connecting their contents with biomedical knowledge. Conversion engines can target the same model, integration layers can preserve the distinction between observations and interpretations, and SPARQL studies can examine medically relevant questions with traceable source evidence. The clinical scenarios in Section 7.3 remain future work: the VCF-side representation they would build on is in place and checked, while the biomedical models and reviewed instance mappings they equally require are not.

## Acknowledgments

Project funding provided from the Research Foundation – Flanders (FWO) (SB Fellowship 1S27825N). Ruben Taelman is a postdoctoral fellow of the Research Foundation – Flanders (FWO) (1202124N).

## Declaration on Generative AI

OpenAI Codex and Claude assisted with preparatory work on the vocabulary, assessment artifacts and manuscript. ChatGPT assisted with restructuring, prose revision, reference checks and prospective query examples.

## References

- [1] C. Gaudet-Blavignac, J. L. Raisaro, V. Touré, S. Österle, K. Crameri, C. Lovis, A national, semantic-driven, three-pillar strategy to enable health data secondary usage interoperability for research within the Swiss Personalized Health Network: Methodological study, *JMIR Medical Informatics* 9 (2021) e27591. URL: <https://medinform.jmir.org/2021/6/e27591/>. doi:10.2196/27591.

- [2] T. E. Putman, K. Schaper, N. Matentzoglou, V. P. Rubinetti, F. S. Alquaddoomi, C. Cox, J. H. Caufield, G. Elsarboukh, S. Gehrke, H. Hegde, J. T. Reese, I. Braun, R. M. Bruskiewich, L. Cappelletti, S. Carbon, A. R. Caron, L. E. Chan, C. G. Chute, K. G. Cortes, V. De Souza, T. Fontana, N. L. Harris, E. L. Hartley, E. Hurwitz, J. O. B. Jacobsen, M. Krishnamurthy, B. J. Laraway, J. A. McLaughlin, J. A. McMurry, S. A. T. Moxon, K. R. Mullen, S. T. O'Neil, K. A. Shefchek, R. Stefancsik, S. Toro, N. A. Vasilevsky, R. L. Walls, P. L. Whetzel, D. Osumi-Sutherland, D. Smedley, P. N. Robinson, C. J. Mungall, M. A. Haendel, M. C. Munoz-Torres, The Monarch Initiative in 2024: an analytic platform integrating phenotypes, genes and diseases across species, *Nucleic Acids Research* 52 (2024) D938–D949. URL: <https://academic.oup.com/nar/article/52/D1/D938/7449493>. doi:10.1093/nar/gkad1082.
- [3] B. Smith, M. Ashburner, C. Rosse, J. Bard, W. Bug, W. Ceusters, L. J. Goldberg, K. Eilbeck, A. Ireland, C. J. Mungall, The OBI Consortium, N. Leontis, P. Rocca-Serra, A. Ruttenberg, S.-A. Sansone, R. H. Scheuermann, N. Shah, P. L. Whetzel, S. Lewis, The OBO Foundry: coordinated evolution of ontologies to support biomedical data integration, *Nature Biotechnology* 25 (2007) 1251–1255. URL: <https://www.nature.com/articles/nbt1346>. doi:10.1038/nbt1346.
- [4] F. Belleau, M.-A. Nolin, N. Tourigny, P. Rigault, J. Morissette, Bio2RDF: Towards a mashup to build bioinformatics knowledge systems, *Journal of Biomedical Informatics* 41 (2008) 706–716. URL: <https://www.sciencedirect.com/science/article/pii/S1532046408000415>. doi:10.1016/j.jbi.2008.03.004.
- [5] P. Danecek, A. Auton, G. Abecasis, C. A. Albers, E. Banks, M. A. DePristo, R. E. Handsaker, G. Lunter, G. T. Marth, S. T. Sherry, G. McVean, R. Durbin, 1000 Genomes Project Analysis Group, The variant call format and VCFtools, *Bioinformatics* 27 (2011) 2156–2158. URL: <https://academic.oup.com/bioinformatics/article/27/15/2156/402296>. doi:10.1093/bioinformatics/btr330.
- [6] VCF, The Variant Call Format Specification: VCFv4.5 and BCFv2.2, 2026. URL: <https://samtools.github.io/hts-specs/VCFv4.5.pdf>, 25 February 2026 printing, version e821e4f; accessed 6 September 2026.
- [7] M. D. Wilkinson, M. Dumontier, I. J. Aalbersberg, G. Appleton, M. Axton, A. Baak, N. Blomberg, J.-W. Boiten, L. B. da Silva Santos, P. E. Bourne, J. Bouwman, A. J. Brookes, T. Clark, M. Crosas, I. Dillo, O. Dumon, S. Edmunds, C. T. Evelo, R. Finkers, A. Gonzalez-Beltran, A. J. G. Gray, P. Groth, C. Goble, J. S. Grethe, J. Heringa, P. A. C. 't Hoen, R. Hooft, T. Kuhn, R. Kok, J. Kok, S. J. Lusher, M. E. Martone, A. Mons, A. L. Packer, B. Persson, P. Rocca-Serra, M. Roos, R. van Schaik, S.-A. Sansone, E. Schultes, T. Sengstag, T. Slater, G. Strawn, M. A. Swertz, M. Thompson, J. van der Lei, E. van Mulligen, J. Velterop, A. Waagmeester, P. Wittenburg, K. Wolstencroft, J. Zhao, B. Mons, The FAIR guiding principles for scientific data management and stewardship, *Scientific Data* 3 (2016) 160018. URL: <https://www.nature.com/articles/sdata201618>. doi:10.1038/sdata.2016.18.
- [8] C. Bahim, C. Casorrán-Amilburu, M. Dekkers, E. Herczog, N. Loozen, K. Repanas, K. Russell, S. Stall, The FAIR Data Maturity Model: An approach to harmonise FAIR assessments, *Data Science Journal* 19 (2020) 41. URL: <https://datascience.codata.org/articles/dsj-2020-041>. doi:10.5334/dsj-2020-041.
- [9] S. Decker, S. Melnik, F. van Harmelen, D. Fensel, M. Klein, J. Broekstra, M. Erdmann, I. Horrocks, The semantic web: the roles of XML and RDF, *IEEE Internet Computing* 4 (2000) 63–73. URL: <https://ieeexplore.ieee.org/document/877487/>. doi:10.1109/4236.877487.
- [10] C. Bizer, T. Heath, T. Berners-Lee, Linked Data—the story so far, *International Journal on Semantic Web and Information Systems* 5 (2009) 1–22. URL: <https://doi.org/10.4018/jswis.2009081901>. doi:10.4018/jswis.2009081901.
- [11] I. M. Lubin, N. Aziz, L. J. Babb, D. Ballinger, H. Bisht, D. M. Church, S. Cordes, K. Eilbeck, F. Hyland, L. Kalman, M. Landrum, E. R. Lockhart, D. Maglott, G. Marth, J. D. Pfeifer, H. L. Rehm, S. Roy, Z. Tezak, R. Truty, M. Ullman-Cullere, K. V. Voelkerding, E. A. Worthey, A. W. Zaranek, J. M. Zook, Principles and recommendations for standardizing the use of the next-generation sequencing variant file in clinical settings, *The Journal of Molecular Diagnostics* 19 (2017) 417–426. URL: <https://doi.org/10.1016/j.jmoldx.2016.12.001>. doi:10.1016/j.jmoldx.2016.12.001.

- [12] J. Baran, B. S. B. Durgahee, K. Eilbeck, E. Antezana, R. Hoehndorf, M. Dumontier, GFVO: the Genomic Feature and Variation Ontology, *PeerJ* 3 (2015) e933. URL: <https://peerj.com/articles/933/>. doi:10.7717/peerj.933.
- [13] E. D. S. Penha, E. Iriabho, A. Dussaq, D. M. de Oliveira, J. S. Almeida, Isomorphic semantic mapping of variant call format (VCF2RDF), *Bioinformatics* 33 (2017) 547–548. URL: <https://academic.oup.com/bioinformatics/article/33/4/547/2593587>. doi:10.1093/bioinformatics/btw652.
- [14] M. Cazzaro, I. G. Gut, L. Menotti, M. Rueda, G. Silvello, HERO-Genomics: An ontology for integration and access of multicenter genomic data, in: *Proceedings of the 16th International Conference on Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2025)*, volume 4196 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2025, pp. 33–42. URL: [https://ceur-ws.org/Vol-4196/paper\\_17.pdf](https://ceur-ws.org/Vol-4196/paper_17.pdf).
- [15] S. Kawashima, T. Fujisawa, T. Katayama, Genome variation ontology for annotation of complex structural variations, in: *Proceedings of the 14th International Conference on Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2023)*, volume 3415 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2023, pp. 126–127. URL: <https://ceur-ws.org/Vol-3415/paper-22.pdf>.
- [16] A. H. Wagner, L. Babb, G. Alterovitz, M. Baudis, M. Brush, D. L. Cameron, M. Cline, M. Griffith, O. L. Griffith, S. E. Hunt, D. Kreda, J. M. Lee, S. Li, J. Lopez, E. Moyer, T. Nelson, R. Y. Patel, K. Riehle, P. N. Robinson, S. Rynearson, H. Schuilenburg, K. Tsukanov, B. Walsh, M. Konopko, H. L. Rehm, A. D. Yates, R. R. Freimuth, R. K. Hart, The GA4GH Variation Representation Specification: A computational framework for variation representation and federated identification, *Cell Genomics* 1 (2021) 100027. doi:10.1016/j.xgen.2021.100027.
- [17] E. Crum, VCF Core Vocabulary, Version 2.1.2. GitHub repository, 2026. URL: <https://github.com/ecrum19/vcf-core-vocabulary/tree/3f06d18>, version 2.1.2 with the SSSOM alignment set; source snapshot of 14 September 2026.
- [18] VCF-RDFizer maintainers, VCF-RDFizer, Version 3.0.3. Open-source software, 2026. URL: <https://github.com/ecrum19/VCF-RDFizer/tree/be658a2>, tag v3.0.3; converted and validated 14 September 2026.
- [19] World Wide Web Consortium, PROV-O: The PROV Ontology, W3C Recommendation, World Wide Web Consortium, 2013. URL: <https://www.w3.org/TR/2013/REC-prov-o-20130430/>.
- [20] K. Eilbeck, S. E. Lewis, C. J. Mungall, M. Yandell, L. Stein, R. Durbin, M. Ashburner, The sequence ontology: a tool for the unification of genome annotations, *Genome Biology* 6 (2005) R44. URL: <https://link.springer.com/article/10.1186/gb-2005-6-5-r44>. doi:10.1186/gb-2005-6-5-r44.
- [21] J. T. Bolleman, C. J. Mungall, F. Strozzi, J. Baran, M. Dumontier, R. J. P. Bonnal, R. Buels, R. Hoehndorf, T. Fujisawa, T. Katayama, P. J. A. Cock, FALDO: a semantic standard for describing the location of nucleotide and protein feature annotation, *Journal of Biomedical Semantics* 7 (2016) 39. URL: <https://link.springer.com/article/10.1186/s13326-016-0067-z>. doi:10.1186/s13326-016-0067-z.
- [22] SAM/BAM Format Specification Working Group, HTS format specifications: VCF specifications and earlier versions, Official specification index, 2026. URL: <https://samtools.github.io/hts-specs/>, accessed 9 September 2026.
- [23] S. Beier, A. Fiebig, C. Pommier, I. Liyanage, M. Lange, P. J. Kersey, S. Weise, R. Finkers, B. Koylass, T. Cezard, M. Courtot, B. Contreras-Moreira, G. Naamati, S. Dyer, U. Scholz, Recommendations for the formatting of variant call format (VCF) files to make plant genotyping data FAIR, *F1000Research* 11 (2022) 231. URL: <https://doi.org/10.12688/f1000research.109080.2>. doi:10.12688/f1000research.109080.2, version 2; peer review: 2 approved.
- [24] Global Alliance for Genomics and Health, A federated ecosystem for sharing genomic, clinical data, *Science* 352 (2016) 1278–1280. doi:10.1126/science.aaf6162.
- [25] Global Alliance for Genomics and Health, HTS Specifications: SAM/BAM and Related High-Throughput Sequencing File Formats, <https://github.com/samtools/hts-specs>, ??? Repository containing the Variant Call Format specifications, including VCF versions 4.1–4.5.

- [26] N. Matentzoglou, J. P. Balhoff, S. M. Bello, C. Bizon, M. Brush, T. J. Callahan, C. G. Chute, W. D. Duncan, C. T. Evelo, D. Gabriel, J. Graybeal, A. Gray, B. M. Gyori, M. Haendel, H. Harmse, N. L. Harris, I. Harrow, H. B. Hegde, A. L. Hoyt, C. T. Hoyt, D. Jiao, E. Jiménez-Ruiz, S. Jupp, H. Kim, S. Koehler, T. Liener, Q. Long, J. Malone, J. A. McLaughlin, J. A. McMurtry, S. Moxon, M. C. Muñoz-Torres, D. Osumi-Sutherland, J. A. Overton, B. Peters, T. Putman, N. Queralt-Rosinach, K. Shefchek, H. Solbrig, A. Thessen, T. Tudorache, N. Vasilevsky, A. H. Wagner, C. J. Mungall, A simple standard for sharing ontological mappings (SSSOM), Database 2022 (2022) baac035. URL: <https://academic.oup.com/database/article/doi/10.1093/database/baac035/6591806>. doi:10.1093/database/baac035.
- [27] World Wide Web Consortium, SKOS Simple Knowledge Organization System Reference, W3C Recommendation, World Wide Web Consortium, 2009. URL: <https://www.w3.org/TR/skos-reference/>.
- [28] SWAT4HCLS 2025 BioHackathon genomic variation participants, Genomic variant RDF representations: mapping declarations to GA4GH VRS, Project site and report, 2025. URL: <https://swat4hcls-2025-genomic-variation.github.io/genomic-variant-schema/>, declarations transcribed as SSSOM sets and republished with the VCF Core Vocabulary; accessed 8 September 2026.
- [29] World Wide Web Consortium, Shapes Constraint Language (SHACL), W3C Recommendation, World Wide Web Consortium, 2017. URL: <https://www.w3.org/TR/2017/REC-shacl-20170720/>.
- [30] C. R. Lee, J. A. Luzum, K. Sangkuhl, R. S. Gammal, M. S. Sabatine, C. M. Stein, D. F. Kisor, N. A. Limdi, Y. M. Lee, S. A. Scott, J.-S. Hulot, D. M. Roden, A. Gaedigk, K. E. Caudle, T. E. Klein, J. A. Johnson, A. R. Shuldiner, Clinical pharmacogenetics implementation consortium guideline for CYP2C19 genotype and clopidogrel therapy: 2022 update, Clinical Pharmacology & Therapeutics 112 (2022) 959–967. doi:10.1002/cpt.2526.
- [31] PharmCAT contributors, PharmCAT methods, Official documentation, 2026. URL: <https://pharmcat.org/methods/>, accessed 9 September 2026.
- [32] Human Phenotype Ontology Consortium, The human phenotype ontology, Project website and documentation, 2026. URL: <https://hpo.jax.org/>, accessed 9 September 2026.
- [33] M. Griffith, N. C. Spies, K. Krysiak, J. F. McMichael, A. C. Coffman, A. M. Danos, B. J. Ainscough, C. A. Ramirez, D. T. Rieke, L. Kujan, E. K. Barnell, A. H. Wagner, Z. L. Skidmore, A. Wollam, C. J. Liu, M. R. Jones, R. L. Bilski, R. Lesurf, Y.-Y. Feng, N. M. Shah, M. Bonakdar, L. Trani, M. Matlock, A. Ramu, K. M. Campbell, G. C. Spies, A. P. Graubert, K. Gangavarapu, J. M. Eldred, D. E. Larson, J. R. Walker, B. M. Good, C. Wu, A. I. Su, R. Dienstmann, A. A. Margolin, D. Tamborero, N. Lopez-Bigas, S. J. M. Jones, R. Bose, D. H. Spencer, L. D. Wartman, R. K. Wilson, E. R. Mardis, O. L. Griffith, CIViC is a community knowledgebase for expert crowdsourcing the clinical interpretation of variants in cancer, Nature Genetics 49 (2017) 170–174. doi:10.1038/ng.3774.
- [34] A. Bodrug-Schepers, H. Chabane, G. Montoya, P. Serrano-Alvarado, R. Redon, A. Gaignard, Semantic beacons: a framework to support federated querying over genomic variants and public knowledge graphs, in: Proceedings of the 16th International Conference on Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2025), volume 4196 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2025, pp. 21–32. URL: [https://ceur-ws.org/Vol-4196/paper\\_16.pdf](https://ceur-ws.org/Vol-4196/paper_16.pdf).